

Digital Signal Processing: A Technical Primer

September 2026

Table of Contents

1	Introduction	4
1.1	What This Primer Covers	4
1.2	What This Primer Is Not	5
1.3	Assumed Background	5
1.4	A Mental Model: DSP as Controlled Information Flow	5
2	Signals, Systems, and Convolution	6
2.1	What Is a Signal?	6
2.2	Common Discrete-Time Operations	7
2.3	Systems and Their Properties	7
2.4	The Unit Impulse and the Impulse Response	8
2.5	Convolution as a Sliding Weighted Sum	9
2.6	Correlation and Matched Similarity	9
2.7	Difference Equations	10
2.8	Causality, Stability, and Poles in the Time Domain	10
3	Fourier Representations and Frequency Response	10
3.1	Complex Exponentials Are Eigenfunctions of LTI Systems	11
3.2	The Discrete-Time Fourier Transform	11
3.3	Fourier Series and the Continuous-Time Transform	12
3.4	Time and Frequency Shifts	12
3.5	Differentiation and Integration	13
3.6	Why the Moving Average Is Not an Ideal Filter	13
4	Sampling, Reconstruction, and Acquisition Limits	13
4.1	Sampling Is Multiplication by an Impulse Train	14
4.2	What Sampling Does in Frequency	14
4.3	The Sampling Theorem, Precisely Enough	15
4.4	Aliasing Is a Design Failure Once It Happens	15
4.5	Reconstruction and the Zero-Order Hold	16
4.6	Quantisation and Signal-to-Quantisation-Noise Ratio	16
4.7	Clock Jitter and Aperture Effects	16
5	The DFT, FFT, and Fast Convolution	17
5.1	Orthogonality of the DFT Basis	17

5.2	The DFT	17
5.3	Conjugate Symmetry and One-Sided Spectra	18
5.4	Parseval's Theorem	18
5.5	Windowing and Leakage	19
5.6	Spectral Leakage Worked Example	19
5.7	Why the Naive DFT Is Expensive	20
5.8	Radix-2 Decimation in Time	20
5.9	Practical FFT Details	21
5.10	Circular Versus Linear Convolution	21
5.11	FFT Numerical Error and Performance	22
5.12	Parseval, Inner Products, and Matched Filters	22
5.13	Toeplitz and Circulant Operators	22
6	The Z-Transform, Poles, Zeros, and Stability	23
6.1	Definition and Region of Convergence	23
6.2	Transfer Functions, Poles, and Zeros	23
6.3	Difference Equations from $H(z)$	24
6.4	Resonators and a Pole Pair	24
6.5	Minimum Phase and Invertibility	24
7	Designing Digital Filters	25
7.1	FIR Linear Phase	25
7.2	Start with a Specification	25
7.3	FIR Versus IIR	26
7.4	Windowed-Sinc FIR Design	26
7.5	Equiripple and Least-Squares FIR Design	27
7.6	Phase, Group Delay, and Transients	28
7.7	Analog Prototypes and IIR Designs	28
7.8	Notches and Resonators Directly in the z -Plane	29
8	Implementing Filters: State, Precision, and Real-Time Constraints	29
8.1	Direct Forms and Biquad Cascades	29
8.2	State Is Part of the Filter	30
8.3	Finite Precision and Internal Dynamic Range	30
8.4	Offline Zero-Phase Filtering	31
8.5	Real-Time Constraints Are Deadlines	31
8.6	Fixed-Point Representation	32
8.7	Quantisation of Filter Coefficients	32
8.8	SIMD, Vectorisation, and Memory Layout	32
8.9	Block Processing and Partitioned Convolution	33
8.10	Numerical Summation	33
9	Sampling-Rate Conversion and Filter Banks	33
9.1	Decimation and Interpolation	33
9.2	Rational Conversion	34
9.3	Multistage Conversion	35

9.4	Filter Banks	35
10	Random Signals and Spectral Measurement	35
10.1	Processes and Realisations	35
10.2	Power Spectral Density	36
10.3	Noise Bandwidth	37
10.4	What a Spectrum Estimate Means	37
10.5	Periodogram Bias and Variance	37
10.6	Welch’s Method	38
10.7	Multitaper and Parametric Alternatives	38
10.8	Coherence and Cross-Spectra	39
11	Estimation, Detection, and Inverse Problems	39
11.1	Wiener Filtering	39
11.2	Matched Filtering	40
11.3	Deconvolution	40
11.4	Least-Mean-Squares Adaptation	40
11.5	System Identification	41
11.6	Stable Inversion as Regularised Projection	41
12	Applications: Time-Varying, Complex, and Spatial Signals	41
12.1	The Short-Time Fourier Transform	41
12.2	Analytic Signals and the Hilbert Transform	42
12.3	Example: Chirps and Transients	42
12.4	Why Complex Signals Appear	43
12.5	Modulation as Frequency Translation	43
12.6	Amplitude, Phase, and Frequency Modulation	44
12.7	Matched Filtering and Detection Thresholds	44
12.8	IQ Imbalance and Image Rejection	45
12.9	Two-Dimensional Convolution	45
12.10	The 2-D Fourier Transform	45
12.11	Sampling in Space	46
13	Building and Verifying a DSP Pipeline	46
13.1	First Questions	46
13.2	Method Selection Heuristics	46
13.3	A DSP Error Budget	47
13.4	Properties Worth Testing	47
13.5	Reproducibility	48
13.6	Common Failure Patterns	48
13.7	Generate a Controlled Test Signal	49
13.8	Design and Apply a Notch	49
13.9	Estimate and Integrate the PSD	50
13.10	Validate Against a Causal Block Path	50
14	Reference Appendix	51

14.1 Complex Arithmetic Without Mysticism	51
14.2 Decibels and Reference Quantities	51
14.3 Frequency Units	52
14.4 Converting Ripple and Attenuation to Linear Values	52
14.5 FIR Order Intuition	52
14.6 IIR Order and Warp Checks	53
14.7 Filter Diagnostics	53
15 Further Reading and Resources	54

1 Introduction

Digital signal processing (DSP) is often introduced as a bag of transforms: take a Fourier transform, multiply by a filter response, and transform back. That description is mathematically correct and practically incomplete. Real DSP is the discipline of turning a physical measurement into a sequence of numbers, deciding what information matters, transforming or estimating that information, and producing an output whose timing, phase, noise, and numerical error are acceptable. The Fourier transform is one of its most useful coordinate systems, not the whole subject.

This primer builds a working model from first principles. It starts with what a signal and a system mean, derives the consequences of sampling, and then develops convolution, Fourier and z -domain analysis, digital filter design, multirate processing, spectral estimation, and common adaptive and time-frequency methods. It connects the mathematics to implementation details: finite records, windows, latency, state, floating-point arithmetic, fixed-point overflow, and the difference between an attractive plot and a correct measurement.

1.1 What This Primer Covers

You will learn how a continuous-time waveform becomes a discrete-time sequence; why the Nyquist rate is a statement about bandwidth rather than a magic property of a device; how linear time-invariant systems are completely described by an impulse response; and why convolution in time becomes multiplication in frequency. You will derive the discrete Fourier transform (DFT), see why the fast Fourier transform (FFT) is an algorithm rather than a different transform, and use the z -transform to reason about poles, zeros, causality, and stability.

The filter sections cover finite impulse response (FIR) and infinite impulse response (IIR) structures, design from frequency specifications, phase and group delay, bilinear-transform frequency warping, numerical implementation with second-order sections, and the choices

involved in real-time versus offline filtering. Later sections cover decimation and interpolation, power spectral density, Welch estimation, matched and adaptive filtering, analytic signals, short-time Fourier analysis, image processing, and an end-to-end Python workflow using NumPy and SciPy.

1.2 What This Primer Is Not

This is not a replacement for a signals-and-systems or DSP textbook. It does not prove every convergence theorem, derive every window's closed-form transform, or give a complete treatment of communication theory, control, radar, biomedical instrumentation, or audio perception. Those subjects use the machinery developed here but deserve primers of their own. It is also not a catalogue of library calls. A library can design a filter in one line; it cannot decide whether the cutoff should be specified before or after decimation, whether phase distortion is acceptable, or whether the spectrum you plotted is scaled correctly.

1.3 Assumed Background

You should be comfortable with algebra, complex numbers, functions, and basic calculus. You need to know what a vector and matrix are, but not to have taken an electrical-engineering course. The text introduces the notation it uses. A small amount of probability helps for the sections on noise and spectral estimation. The code examples use Python with NumPy and SciPy; the concepts apply equally to MATLAB, R, Julia, C, and embedded DSP libraries.

1.4 A Mental Model: DSP as Controlled Information Flow

Keep this pipeline in mind:

```
physical phenomenon
  ↓ sensor and analogue conditioning
continuous-time voltage/current or transducer output
  ↓ anti-aliasing filter + sample-and-hold + quantiser
finite-precision discrete-time sequence  $x[n]$ 
  ↓ algorithm with state, parameters, and latency
discrete-time sequence  $y[n]$ 
  ↓ optional reconstruction and analogue conditioning
actuator, display, storage, or decision
```

Every arrow can lose information or introduce error. A low-pass filter before an analogue-to-digital converter (ADC) prevents frequencies above the usable band from folding into it. A quantiser replaces a continuum of amplitudes with a finite set of codes. A causal filter cannot use samples that have not arrived yet, so phase and latency are design constraints rather than afterthoughts. A spectrum computed from a finite record is an estimate with a window, not a direct photograph of an infinite signal.

The central DSP question is therefore not “which transform should I call?” It is “what information is present, what information do I need, and what operations preserve or deliberately discard it?”

2 Signals, Systems, and Convolution

Start with the objects DSP operates on: sequences, systems, impulses, recurrences, and the convolution and correlation operations that connect them.

2.1 What Is a Signal?

A signal is a quantity that varies over an independent variable and carries information. For a microphone, the independent variable is time and the signal may be acoustic pressure or voltage. For an image, the independent variables are two spatial coordinates. For a sensor network, the signal may be indexed by time and sensor number. DSP usually denotes a continuous-time signal by $x(t)$ and a discrete-time sequence by $x[n]$, where n is an integer sample index.

The distinction is important. A discrete-time signal is indexed only at integer positions; it is not automatically quantised in amplitude. A sequence such as

$$x[n] = \sin(0.2\pi n)$$

has discrete time but real-valued amplitude. A digital signal normally has both discrete time and finite-precision amplitude because a computer or converter stores a finite representation. Keeping time discretisation and amplitude discretisation separate prevents many sloppy explanations of sampling.

Signals can be classified along several axes. A deterministic signal is specified by a rule or known record; a stochastic process is described by probability distributions and statistical properties. A periodic sequence repeats exactly, while an aperiodic sequence does not. An energy signal has finite total energy,

$$E_x = \sum_{n=-\infty}^{\infty} |x[n]|^2 < \infty,$$

whereas a power signal has finite nonzero average power,

$$P_x = \lim_{N \rightarrow \infty} \frac{1}{2N+1} \sum_{n=-N}^N |x[n]|^2.$$

The categories overlap imperfectly in practical finite records. A finite audio clip is an energy signal in the mathematical model; a stationary noise process is usually discussed through its power spectral density. The classification tells you which normalisation and convergence ideas are appropriate.

2.2 Common Discrete-Time Operations

The basic operations are simple but form the vocabulary of every later derivation:

- **Shift:** $x[n - n_0]$ delays a sequence by n_0 samples; $x[n + n_0]$ advances it.
- **Reversal:** $x[-n]$ reflects the sequence around $n = 0$.
- **Scaling:** $Ax[n]$ changes amplitude; $x[Mn]$ keeps every M th sample and is decimation without anti-alias filtering.
- **Expansion:** $x[n/L]$ when n is a multiple of L , with zeros elsewhere, is upsampling by L .
- **Difference:** $x[n] - x[n - 1]$ emphasises changes and approximates a derivative after scaling by the sample interval.
- **Running sum:** $\sum_{k=-\infty}^n x[k]$ approximates integration and is the discrete-time accumulator.

The notation $x[2n]$ does not mean “make the signal twice as fast” in isolation. It selects an even-indexed subsequence and changes the sampling rate’s interpretation only when a physical sample interval is attached to the sequence.

2.3 Systems and Their Properties

A system maps an input signal to an output: $y = \mathcal{T}\{x\}$. Four properties matter constantly.

- **Linearity:** $\mathcal{T}\{ax_1 + bx_2\} = a\mathcal{T}\{x_1\} + b\mathcal{T}\{x_2\}$.
- **Time invariance:** delaying the input delays the output by the same amount. If $y[n] = \mathcal{T}\{x[n]\}$, then $\mathcal{T}\{x[n - n_0]\} = y[n - n_0]$.
- **Causality:** $y[n]$ depends only on samples at or before n . A noncausal system can use future samples and is possible for an offline record, but not for a live stream without adding delay.
- **Stability:** in the bounded-input, bounded-output (BIBO) sense, every bounded input produces a bounded output.

Linear time-invariant (LTI) systems are the centre of classical DSP because they are both expressive and analyzable. A nonlinear or time-varying system can be useful, but its behaviour cannot be inferred from one impulse response and one frequency-response curve.

2.4 The Unit Impulse and the Impulse Response

The discrete-time unit impulse is

$$\delta[n] = \begin{cases} 1, & n = 0, \\ 0, & n \neq 0. \end{cases}$$

Every sequence can be decomposed into shifted, scaled impulses:

$$x[n] = \sum_{k=-\infty}^{\infty} x[k]\delta[n-k].$$

If an LTI system's response to $\delta[n]$ is $h[n]$, linearity and time invariance give its response to $x[n]$:

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k].$$

The sequence $h[n]$ is the impulse response. This is not merely a convenient formula. It says that an LTI system is completely characterised by how it responds to one impulse, provided the relevant sums converge. A short $h[n]$ gives a finite impulse response filter; an impulse response that continues forever gives an infinite impulse response filter.

For a causal filter, $h[n] = 0$ for $n < 0$, and the convolution can be evaluated online as

$$y[n] = \sum_{k=0}^{\infty} h[k]x[n-k].$$

If $h[n]$ is absolutely summable,

$$\sum_{n=-\infty}^{\infty} |h[n]| < \infty,$$

the filter is BIBO stable. A practical FIR is automatically stable; an IIR requires its poles to lie inside the unit circle for the usual causal realisation.

2.5 Convolution as a Sliding Weighted Sum

For a finite impulse response $h[0], \dots, h[M]$, convolution is a dot product between the current input window and the reversed coefficient vector:

$$y[n] = h[0]x[n] + h[1]x[n-1] + \dots + h[M]x[n-M].$$

The coefficients are not “the frequencies to keep.” They are time-domain weights whose collective response determines the frequency behaviour. A moving average with M taps,

$$h[k] = \frac{1}{M}, \quad 0 \leq k < M,$$

reduces rapid variation because neighbouring samples are averaged. It also delays the signal by approximately $(M-1)/2$ samples if interpreted as a linear-phase FIR, attenuates some frequencies, and introduces a frequency response with sidelobes. The smoothing is never free.

Convolution is commutative and associative where the sums exist:

$$x * h = h * x, \quad (x * h) * g = x * (h * g).$$

This lets you combine cascaded LTI filters into one equivalent impulse response. In implementation, however, direct convolution may be a poor choice for long filters: FFT-based overlap-save or polyphase structures can reduce the cost.

2.6 Correlation and Matched Similarity

Correlation compares one sequence with shifted versions of another. A common convention for cross-correlation is

$$r_{xy}[\ell] = \sum_n x^*[n]y[n+\ell],$$

where $*$ denotes complex conjugation. Autocorrelation sets $x = y$ and measures similarity with shifted copies of the same signal. Correlation is closely related to convolution: cross-correlation is convolution with a time-reversed conjugated sequence.

This distinction matters in detection. A matched filter uses a time-reversed, conjugated copy of the expected waveform, so the output peaks when the waveform is aligned. Calling every correlation a convolution produces the wrong phase and often the wrong lag sign.

2.7 Difference Equations

Many digital filters are described by a linear constant-coefficient difference equation:

$$y[n] + a_1y[n-1] + \cdots + a_Ny[n-N] = b_0x[n] + b_1x[n-1] + \cdots + b_Mx[n-M].$$

The b_k coefficients form the feed-forward path and the a_k coefficients form feedback. If all feedback coefficients are zero, the filter is FIR. If feedback is present, the impulse response generally continues indefinitely and the filter is IIR.

The equation is a computational specification only after initial conditions are defined. For a stream, the previous input and output states come from the preceding block. For a finite record, setting all states to zero creates a startup transient; setting them to a steady-state value can reduce the transient for particular inputs. A block implementation that silently resets state at every block is not equivalent to one continuous stream.

2.8 Causality, Stability, and Poles in the Time Domain

An IIR recurrence can look harmless and still be unstable. The first-order system

$$y[n] = 0.99y[n-1] + x[n]$$

is a stable smoother; replacing 0.99 with 1.01 causes the homogeneous response to grow as 1.01^n . The distinction is visible directly from the pole location: the pole is at $z = 0.99$ or $z = 1.01$.

In a finite-precision implementation, a pole theoretically inside the unit circle can still behave badly if coefficient quantisation moves it outside, if internal states overflow, or if denormal values cause unacceptable performance. Stability analysis must cover both the mathematical transfer function and the chosen realisation.

3 Fourier Representations and Frequency Response

Once the time-domain model is in place, Fourier representations provide a second coordinate system. The same operations can then be understood as changes in amplitude, phase, and frequency.

3.1 Complex Exponentials Are Eigenfunctions of LTI Systems

For an LTI system with frequency response $H(e^{j\omega})$, a complex exponential input $x[n] = e^{j\omega n}$ produces

$$y[n] = \sum_k h[k] e^{j\omega(n-k)} = e^{j\omega n} \sum_k h[k] e^{-j\omega k} = H(e^{j\omega}) e^{j\omega n}.$$

The system changes only amplitude and phase; it does not create new frequencies. This is why frequency response is so useful. A real sinusoid is the sum of positive- and negative-frequency complex exponentials, and real filters preserve the conjugate symmetry needed to produce a real output.

3.2 The Discrete-Time Fourier Transform

The discrete-time Fourier transform (DTFT) of $x[n]$ is

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n},$$

with inverse

$$x[n] = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(e^{j\omega}) e^{j\omega n} d\omega.$$

The DTFT is continuous in ω but periodic with period 2π . Normalized frequency ω is radians per sample. If the sample rate is f_s , physical frequency is $f = \omega f_s / (2\pi)$, and the interval $[-\pi, \pi)$ corresponds to $[-f_s/2, f_s/2)$.

For an LTI system,

$$y[n] = x[n] * h[n] \quad \Longleftrightarrow \quad Y(e^{j\omega}) = X(e^{j\omega}) H(e^{j\omega}).$$

The convolution theorem is the mathematical basis for filtering in the frequency domain. It also explains why a frequency-domain filter is not merely “delete bins”: if X is a finite-record DFT, multiplying by a smooth frequency response corresponds to circular convolution in the finite-dimensional model unless overlap-save or overlap-add is used correctly.

3.3 Fourier Series and the Continuous-Time Transform

The Fourier series represents a periodic continuous-time signal as harmonics of its fundamental frequency:

$$x(t) = \sum_{k=-\infty}^{\infty} c_k e^{jk\omega_0 t}, \quad c_k = \frac{1}{T_0} \int_{T_0} x(t) e^{-jk\omega_0 t} dt.$$

The continuous-time Fourier transform (CTFT) generalises this to aperiodic signals:

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi ft} dt, \quad x(t) = \int_{-\infty}^{\infty} X(f) e^{j2\pi ft} df.$$

The four Fourier representations differ in what is discrete, periodic, finite, or continuous. Confusing them causes unit and scaling errors:

Representation	Time/domain index	Frequency/domain	
		index	Frequency periodic?
Fourier series	continuous and periodic	discrete harmonics	no, discrete set
CTFT	continuous and aperiodic	continuous	no
DTFT	discrete and generally aperiodic	continuous	yes, 2π
DFT	finite discrete record	finite discrete bins	implicit periodic extension

3.4 Time and Frequency Shifts

The DTFT shift property follows directly:

$$x[n - n_0] \iff e^{-j\omega n_0} X(e^{j\omega}).$$

The magnitude is unchanged; the phase gains a linear term. This is why delay can be measured from phase slope, and why a pure delay is all-pass in magnitude. For a noninteger delay, the ideal response is $e^{-j\omega D}$, but a finite causal filter can only approximate it over a specified band.

Modulation and convolution form a dual pair. Multiplying a signal by a sinusoid shifts its spectrum; convolving in time with a short pulse smooths or shapes the spectrum according to the pulse’s transform. These properties let you predict a filter’s output before simulating it.

3.5 Differentiation and Integration

The backward difference filter has

$$H(e^{j\omega}) = 1 - e^{-j\omega} = 2je^{-j\omega/2} \sin(\omega/2).$$

At low frequencies its magnitude is approximately $|\omega|$, so it approximates a derivative. At high frequencies it amplifies relative high-frequency content and has a zero at DC. The accumulator has a pole at $z = 1$, so it integrates but is not BIBO stable: a bounded nonzero-mean input can produce an unbounded ramp.

In numerical work, differentiation amplifies measurement noise and integration accumulates bias. A high-pass filter or detrending may be needed before differentiation; leakage control, drift removal, or a physical reference may be needed before integration. Choosing a discrete approximation without considering the noise spectrum is a common source of false dynamics.

3.6 Why the Moving Average Is Not an Ideal Filter

For a length- M moving average,

$$H(e^{j\omega}) = \frac{1}{M} \sum_{k=0}^{M-1} e^{-j\omega k} = e^{-j\omega(M-1)/2} \frac{\sin(M\omega/2)}{M \sin(\omega/2)}.$$

The magnitude has a main lobe and sidelobes. It attenuates high frequencies but also passes some stopband frequencies through sidelobes. Increasing M narrows the main lobe and increases delay. This one formula is a useful antidote to the phrase “averaging removes noise”: averaging suppresses components according to a known response, and the response has trade-offs.

4 Sampling, Reconstruction, and Acquisition Limits

Sampling is where the physical signal chain meets discrete-time mathematics. The following chapter treats aliasing, reconstruction, quantisation, and timing error as acquisition con-

straints, not as details to repair later.

4.1 Sampling Is Multiplication by an Impulse Train

Suppose a continuous signal $x(t)$ is sampled every T seconds, at sampling frequency $f_s = 1/T$. An idealised sampled signal is

$$x_s(t) = x(t) \sum_{n=-\infty}^{\infty} \delta(t - nT) = \sum_{n=-\infty}^{\infty} x(nT) \delta(t - nT).$$

The discrete sequence is $x[n] = x(nT)$. A physical ADC is not an ideal impulse-train multiplier: it includes a sample-and-hold, finite aperture, analogue front-end, clock jitter, and quantisation. The ideal model isolates the fundamental question: can the original band-limited signal be reconstructed from its samples?

4.2 What Sampling Does in Frequency

Multiplication in time becomes convolution in frequency. The Fourier transform of the impulse train is another impulse train, so sampling replicates the continuous-time spectrum every f_s :

$$X_s(f) = \frac{1}{T} \sum_{k=-\infty}^{\infty} X(f - kf_s).$$

If the replicas do not overlap, an analogue low-pass reconstruction filter can select the central copy. If they overlap, the sum in the overlap is irreversible: different continuous-time frequencies have produced the same samples. This is aliasing.

The same fact appears for a sampled sinusoid. Since

$$\cos(2\pi f_0 nT) = \cos\left(2\pi \frac{f_0}{f_s} n\right),$$

frequencies that differ by integer multiples of f_s produce identical discrete-time sequences. Also, f_0 and $-f_0$ produce the same real cosine. Discrete-time frequency is periodic modulo 2π radians per sample.

4.3 The Sampling Theorem, Precisely Enough

If $x(t)$ is strictly band-limited to $|f| < B$ and sampled at $f_s > 2B$, ideal sinc interpolation reconstructs it exactly:

$$x(t) = \sum_{n=-\infty}^{\infty} x[n] \operatorname{sinc}\left(\frac{t - nT}{T}\right), \quad \operatorname{sinc}(u) = \frac{\sin(\pi u)}{\pi u}.$$

The inequality is strict in practical systems because real filters have finite transition bands and real signals are not perfectly band-limited. “Nyquist frequency” means $f_s/2$, the highest unaliased frequency for a baseband sampled system; “Nyquist rate” is twice the highest frequency occupied by the signal. They are not the same phrase.

The theorem is about representation, not about the quality of a reconstruction filter or the absence of noise. It does not say that sampling at just over twice a vaguely defined centre frequency is safe. For a bandpass signal, bandpass sampling can use a lower rate if the spectral replicas are deliberately arranged not to overlap, but the allowable bands and analogue filtering must be engineered.

4.4 Aliasing Is a Design Failure Once It Happens

Consider a 7 kHz tone sampled at 10 kHz. Its aliases appear at frequencies equivalent modulo 10 kHz; in the baseband it appears at $|7 - 10| = 3$ kHz. A later digital low-pass filter can remove a 3 kHz tone only by also removing genuine 3 kHz content. The filter cannot determine whether the sample sequence came from a 3 kHz or 7 kHz analogue sinusoid.

The anti-aliasing filter must therefore be analogue, or must occur before the sampling operation at an earlier, faster sampling rate. In a practical acquisition chain:

1. define the highest signal frequency you care about;
2. select f_s with room for a transition band;
3. design the analogue anti-aliasing filter so out-of-band energy is sufficiently attenuated at $f_s/2$;
4. account for the sensor, amplifier, filter, ADC aperture, clock, and quantisation together;
5. verify the actual spectrum with a test signal and a calibrated measurement path.

Oversampling makes the analogue filter easier: a larger gap between the signal band and Nyquist frequency permits a gentler transition. It does not make aliasing impossible; it moves the engineering boundary.

4.5 Reconstruction and the Zero-Order Hold

The ideal sinc interpolator is noncausal and infinitely long. A digital-to-analogue converter (DAC) commonly holds each sample until the next update, producing a zero-order-hold waveform. In frequency, the hold contributes a sinc-shaped envelope and the staircase contains images at multiples of f_s . An analogue reconstruction filter removes those images and often compensates, approximately, for the hold's passband droop.

When DSP documentation says “the output is a 1 kHz sine,” ask whether it means the discrete sequence has a normalized digital frequency, the DAC output has a 1 kHz analogue component, or the entire reconstruction chain has been characterised. Those are related but not identical statements.

4.6 Quantisation and Signal-to-Quantisation-Noise Ratio

An ideal B -bit uniform quantiser maps an amplitude range into 2^B levels with step size Δ . The error $e[n] = x_q[n] - x[n]$ is bounded approximately by $\pm\Delta/2$ when the input is not overloaded. Under common assumptions—error uncorrelated with the signal and approximately uniform—the error power is $\Delta^2/12$.

For a full-scale sine wave into an ideal B -bit ADC, the familiar approximation is

$$\text{SNR}_q \approx 6.02B + 1.76 \text{ dB}.$$

This is a reference, not a guarantee. Missing codes, nonlinearity, thermal noise, clock jitter, reference noise, and analogue distortion reduce real effective number of bits (ENOB). A low-amplitude signal also uses fewer quantisation levels and therefore has worse signal-to-quantisation-noise ratio relative to its own amplitude. Dither can decorrelate quantisation error from the signal, replacing deterministic distortion with noise whose statistics are easier to model.

4.7 Clock Jitter and Aperture Effects

Sampling time errors matter more at high frequency and high slew rate. If the intended sample time is $t_n = nT$ but the actual time is $t_n + \epsilon_n$, then to first order

$$x(t_n + \epsilon_n) \approx x(t_n) + \epsilon_n x'(t_n).$$

For a sinusoid, timing jitter creates amplitude error proportional to frequency. A useful

approximate jitter-limited signal-to-noise ratio is

$$\text{SNR}_{\text{jitter}} \approx -20 \log_{10}(2\pi f_{\text{in}} \sigma_t),$$

where σ_t is the RMS timing uncertainty. This is why a converter can have enough nominal bits but still fail at radio frequencies: the clock may be the limiting component.

5 The DFT, FFT, and Fast Convolution

The DTFT is continuous in frequency; real programs operate on finite records. This chapter develops the finite transform, its geometry and calibration, the FFT algorithm, and the boundary rules needed for fast convolution.

5.1 Orthogonality of the DFT Basis

Let $W_N = e^{-j2\pi/N}$. The DFT basis vector for bin k has entries W_N^{kn} . For bins k and r ,

$$\sum_{n=0}^{N-1} W_N^{kn} (W_N^{rn})^* = \sum_{n=0}^{N-1} e^{j2\pi(r-k)n/N}.$$

If $r = k$, the sum is N . If $r \neq k$, it is a finite geometric series whose numerator is $1 - e^{j2\pi(r-k)} = 0$ while the denominator is nonzero, so the sum is zero. The DFT coefficient is the projection of the data onto one of these orthogonal basis vectors.

This explains exact bin alignment. A bin-centred sinusoid is orthogonal to every other bin's basis vector over the record. An off-bin sinusoid is not, so its projection is distributed across the basis. Leakage is a geometry problem before it is a window problem.

5.2 The DFT

For a length- N sequence, the DFT is

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, \dots, N-1,$$

with inverse

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j2\pi kn/N}.$$

The DFT treats the input as one period of a periodic sequence. Consequently, the boundary between $x[N-1]$ and $x[0]$ is considered adjacent. If those endpoints do not match, the periodic extension has a jump. That jump contributes energy across the spectrum, producing spectral leakage.

The bin spacing is $\Delta f = f_s/N$. This is the spacing between evaluated frequencies, not automatically the smallest frequency difference the data can resolve. Resolution depends on record duration $T = N/f_s$, window shape, signal-to-noise ratio, and the separation and relative amplitudes of the components. Zero-padding increases the density of plotted samples in the interpolated spectrum; it does not increase the information or the fundamental main-lobe width.

5.3 Conjugate Symmetry and One-Sided Spectra

For real $x[n]$,

$$X[N-k] = X[k]^*.$$

The negative-frequency half is redundant. For even N , the DC bin $k = 0$ and Nyquist bin $k = N/2$ are self-conjugate and real-valued. A one-sided amplitude spectrum normally doubles the magnitude of interior positive-frequency bins but not DC or Nyquist. The exact factor depends on whether the transform is peak amplitude, RMS amplitude, power, or a density per hertz.

For a real signal with a sinusoid of peak amplitude A exactly on a positive-frequency bin, a common one-sided peak-amplitude estimate is

$$A_{\text{estimate}} = \frac{2|X[k]|}{N},$$

with the doubling exception for DC and Nyquist. Off-bin sinusoids and windowing require coherent-gain correction if you want amplitude estimates rather than a qualitative spectrum.

5.4 Parseval's Theorem

With the DFT convention above,

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2.$$

This is a conservation law for energy under the transform, subject to the chosen normalization. It is one of the best validation checks for custom FFT code, window corrections, and power calculations. If a spectrum’s integrated power does not agree with time-domain mean-square power after accounting for the window and normalization, do not trust the plot.

5.5 Windowing and Leakage

Multiplying a finite record by a window $w[n]$ changes its transform from a sharp but leaky rectangular-record spectrum to

$$X_w(e^{j\omega}) = X(e^{j\omega}) * W(e^{j\omega})$$

up to the transform’s normalization. The window’s main lobe controls how close two components can be before they merge; sidelobes control how much a strong component masks nearby weak components.

Common windows encode different trade-offs:

- **Rectangular:** narrowest main lobe for a given record, but high sidelobes. Suitable when the record contains an integer number of cycles and leakage is not a problem.
- **Hann:** good general-purpose reduction of sidelobes with moderate main-lobe widening.
- **Hamming:** lower first sidelobe than rectangular, with a different sidelobe decay pattern.
- **Blackman:** stronger sidelobe suppression at the cost of wider peaks.
- **Kaiser:** adjustable parameter that makes the trade-off explicit.

Use a window because you can state what trade-off you want, not because a plotting recipe says to. For amplitude measurement, correct for coherent gain. For power spectral density, use the window’s mean-square normalization and the sampling rate.

5.6 Spectral Leakage Worked Example

Suppose $f_s = 1000$ Hz and you record $N = 1000$ samples. The bin spacing is 1 Hz. A 50 Hz sine completes exactly 50 cycles and aligns with a bin; a 50.5 Hz sine does not. Under a rectangular window, the second signal is represented by a Dirichlet-kernel-shaped lobe spread across many bins. It has not acquired extra frequencies in the physical world; the finite observation and implicit periodic extension make the basis functions unable to represent its frequency with one coefficient.

The correct response depends on the question. If you need to estimate a known sinusoid’s

amplitude and frequency, fit a sinusoidal model or use an estimator designed for that task. If you need a robust view of broadband noise, use a windowed PSD estimate. If you need to inspect transients, a single long FFT hides their timing and a short-time method is more appropriate.

5.7 Why the Naive DFT Is Expensive

The direct DFT evaluates N sums, each with N terms, for $O(N^2)$ complex operations. For $N = 1,048,576$, that is on the order of a trillion pairwise contributions. The FFT computes the same mathematical DFT in roughly $O(N \log_2 N)$ operations by reusing subexpressions.

5.8 Radix-2 Decimation in Time

For even N , split the input into even and odd samples:

$$X[k] = \sum_{m=0}^{N/2-1} x[2m]e^{-j2\pi k(2m)/N} + \sum_{m=0}^{N/2-1} x[2m+1]e^{-j2\pi k(2m+1)/N}.$$

Define the length- $N/2$ transforms of the even and odd subsequences as $E[k]$ and $O[k]$, and let $W_N^k = e^{-j2\pi k/N}$. Then

$$X[k] = E[k] + W_N^k O[k],$$

$$X[k + N/2] = E[k] - W_N^k O[k], \quad 0 \leq k < N/2.$$

The pair of additions is the butterfly. Recursing until transforms of length one gives $\log_2 N$ stages, each doing N -scale work. This is the Cooley–Tukey decomposition published in 1965; related decompositions were known earlier, but the algorithm made the speedup operationally important.

The FFT is not an approximation to the DFT. Apart from roundoff and implementation details, it produces the same DFT values. A larger FFT does not magically improve the underlying record’s resolution; it only changes how efficiently and densely you evaluate the transform.

5.9 Practical FFT Details

Libraries choose conventions for sign, normalization, array ordering, and treatment of real input. Check them. In NumPy, `fft` returns the unnormalised forward transform and `ifft` applies the $1/N$ scaling; `rfft` exploits Hermitian symmetry for real input. `fftfreq` returns signed bin frequencies, while `rfftfreq` returns the nonnegative frequencies corresponding to `rfft`.

```
import numpy as np

fs = 2_000.0
duration = 1.0
t = np.arange(0.0, duration, 1.0 / fs)
x = 0.7 * np.sin(2 * np.pi * 123.0 * t)

window = np.hanning(len(x))
X = np.fft.rfft(x * window)
f = np.fft.rfftfreq(len(x), d=1.0 / fs)

# Magnitude, corrected for the window's coherent gain.
coherent_gain = window.mean()
amplitude = 2 * np.abs(X) / (len(x) * coherent_gain)
amplitude[[0, -1]] /= 2 # DC and Nyquist are not doubled.
peak_frequency = f[np.argmax(amplitude)]
print(f"peak ~= {peak_frequency:.1f} Hz")
```

The code uses an on-bin-ish sinusoid and a Hann window. If the input frequency is not aligned with a bin, the maximum bin is only a quantised estimate of the peak. Interpolating nearby bins may improve a frequency estimate for a clean sinusoid, but it is not a general cure for leakage.

5.10 Circular Versus Linear Convolution

The inverse DFT of a product of two length- N DFTs is circular convolution:

$$y[n] = \sum_{k=0}^{N-1} x[k]h[(n - k) \bmod N].$$

If you need linear convolution of sequences of lengths L and M , zero-pad both to at least $L + M - 1$ before multiplying their FFTs. For streaming signals, overlap-add and overlap-save divide the input into blocks, manage the boundary samples, and reuse one filter transform. Forgetting this distinction creates wraparound artifacts that can look like instability or ringing.

5.11 FFT Numerical Error and Performance

Floating-point FFTs accumulate roundoff. The error is normally small relative to the transform, but very large dynamic ranges can bury weak components. Use double precision when the noise floor matters, avoid repeated forward/inverse transforms when a single operation will do, and validate with Parseval and known tones.

Performance depends on factorisation, memory layout, cache, SIMD, threading, and whether real-input routines can be used. Powers of two are convenient but not mandatory; modern libraries handle mixed-radix lengths well. Padding to a convenient length can improve speed or visual interpolation, but it changes the transform grid and record length used in scaling.

5.12 Parseval, Inner Products, and Matched Filters

The DFT is a change of orthogonal coordinates. With the unitary normalization, the inner product is preserved:

$$\langle x, y \rangle = \sum_n x[n]y^*[n] = \sum_k X[k]Y^*[k].$$

Correlation with a template is therefore a projection onto shifted template vectors. Matched filtering is not a mysterious “pattern recognizer”; it is a projection weighted according to the noise geometry. If the noise covariance is R , the relevant inner product is $x^H R^{-1} y$. Whitening transforms that inner product back into the ordinary Euclidean one.

5.13 Toeplitz and Circulant Operators

Linear convolution by a fixed finite sequence can be written as multiplication by a Toeplitz matrix. The matrix is banded for a short FIR. Circular convolution is multiplication by a circulant matrix, and the DFT diagonalises every circulant matrix:

$$C = F^{-1} \Lambda F.$$

This is the linear-algebra reason FFT convolution works. The input and filter must be embedded into a large enough circulant system to reproduce the desired Toeplitz multiplication. Padding is not a cosmetic implementation detail; it changes which operator you are applying.

6 The Z-Transform, Poles, Zeros, and Stability

The Fourier transform describes steady-state frequency behaviour. The z -transform adds growth and decay, making causality, transients, stability, and invertibility explicit.

6.1 Definition and Region of Convergence

The bilateral z -transform of $x[n]$ is

$$X(z) = \sum_{n=-\infty}^{\infty} x[n]z^{-n}.$$

The region of convergence (ROC) is the set of complex z values where the sum converges. The same algebraic rational expression can represent different sequences if the ROC differs. For example,

$$\frac{1}{1 - az^{-1}}$$

represents the right-sided sequence $a^n u[n]$ when $|z| > |a|$, and a left-sided sequence $-a^n u[-n - 1]$ when $|z| < |a|$.

The z -transform adds growth or decay to the Fourier analysis through $z = re^{j\omega}$. The DTFT is the z -transform evaluated on the unit circle, $z = e^{j\omega}$, when the ROC includes that circle.

6.2 Transfer Functions, Poles, and Zeros

For an LTI system with zero initial conditions,

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{k=0}^M b_k z^{-k}}{1 + \sum_{k=1}^N a_k z^{-k}}.$$

Zeros are roots of the numerator; poles are roots of the denominator. Evaluating $H(z)$ on the unit circle gives the frequency response. A zero near a unit-circle angle suppresses that frequency. A pole near the unit circle boosts the response and lengthens the transient. The pole-zero plot is therefore a geometric summary of a filter's frequency selectivity and time behaviour.

For a causal rational filter, the ROC is outside the outermost pole. BIBO stability requires the unit circle to lie in the ROC, which for the causal case means all poles are strictly inside

it. A pole exactly on the unit circle is a boundary case: it may produce a bounded response for some inputs but is not BIBO stable.

6.3 Difference Equations from $H(z)$

Starting with

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}},$$

cross-multiplication gives

$$y[n] + a_1 y[n-1] + a_2 y[n-2] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2].$$

The sign convention is worth checking: libraries often store denominator coefficients `[1, a1, a2]` but implement the feedback subtraction implied by moving the delayed output terms to the right-hand side. A sign error can turn a stable low-pass into an unstable oscillator.

6.4 Resonators and a Pole Pair

A complex-conjugate pole pair at radius r and angle ω_0 produces a real resonator. As r approaches 1, the resonance becomes narrower and the ring-down lasts longer. The approximate quality factor grows as the pole approaches the unit circle, but high selectivity raises sensitivity to coefficient quantisation and frequency drift.

This is the same trade-off seen in analogue circuits. The z -plane does not eliminate physics; it gives a convenient coordinate system for discrete-time dynamics.

6.5 Minimum Phase and Invertibility

A causal stable system with all zeros inside the unit circle is minimum phase. Among causal stable systems with a given magnitude response, the minimum-phase version has the smallest group delay in a useful sense. Moving a zero outside the unit circle to its reciprocal conjugate preserves magnitude on the unit circle but changes phase and delay.

Exact inversion is safe only when the inverse is stable and the signal does not contain frequencies where the original response is near zero. Deconvolution that divides by tiny spectral values amplifies noise. Regularisation is often the correct answer: replace a fragile inverse with a controlled estimate.

7 Designing Digital Filters

A filter design is an approximation problem with requirements for magnitude, phase, delay, transients, and resource use. Start with those requirements, then choose FIR or IIR and a design method.

7.1 FIR Linear Phase

A real FIR filter with symmetric coefficients,

$$h[n] = h[M - n], \quad 0 \leq n \leq M,$$

has a frequency response whose phase is linear apart from sign changes. Its group delay is approximately $M/2$ samples. Antisymmetric coefficients provide other linear-phase responses useful for differentiators and Hilbert transformers. Linear phase means all frequencies experience the same delay; it does not mean zero delay.

The causal, exact, zero-phase ideal low-pass filter has a two-sided sinc impulse response and therefore cannot run in real time. Offline forward-backward filtering can cancel phase, but it squares the magnitude response and introduces edge-handling choices. “Zero phase” is a property of an offline procedure, not a free causal filter design.

7.2 Start with a Specification

“Remove the noise” is not a filter specification. A usable specification states the signal band, unwanted band, acceptable gain error, attenuation, phase or delay requirement, sample rate, available computation, and whether the system is causal.

For a low-pass filter, write down at least:

- passband edge f_p ;
- stopband edge f_s (use a different symbol from sampling frequency when documenting; f_{stop} is clearer);
- maximum passband ripple R_p in dB;
- minimum stopband attenuation A_s in dB;
- allowed group delay and transient length;
- sample rate and expected coefficient representation.

The transition width $f_{\text{stop}} - f_p$ controls the order. A narrower transition needs more taps for an FIR or a higher order for an IIR. Increasing stopband attenuation also costs order. If the

specification does not mention the transition band, the design problem is underspecified.

Magnitude and phase are separate requirements. A filter can have an excellent magnitude response and unacceptable waveform distortion. Conversely, an all-pole IIR may meet a magnitude specification at a fraction of the FIR cost while adding frequency-dependent delay. The correct choice follows the signal’s meaning: a detection system may care about matched response and timing, a measurement system may need calibrated amplitude and phase, and a monitoring display may accept substantial delay.

7.3 FIR Versus IIR

An FIR filter computes a weighted finite sum. It is inherently stable for finite coefficients, can have exact linear phase, and is straightforward to restart or parallelise. Its cost is often a long impulse response, especially when the transition band is narrow at a high sample rate. A causal symmetric FIR has a delay of about half its length.

An IIR filter reuses previous outputs. It can achieve sharp magnitude transitions with far fewer multiplications and lower delay, but its feedback makes stability, quantisation sensitivity, startup state, and phase more complicated. High-order IIR filters should almost never be implemented as one polynomial numerator and denominator in floating point, and especially not in a narrow fixed-point format. Use cascaded second-order sections (biquads) and test the actual implementation.

Neither class dominates. A multirate system may use a linear-phase FIR as an anti-alias filter because the rate reduction makes its cost manageable. A low-latency embedded monitor may choose a biquad cascade. An offline scientific pipeline may use forward-backward IIR filtering, accepting its noncausality and changed magnitude response.

7.4 Windowed-Sinc FIR Design

The ideal discrete-time low-pass frequency response with cutoff ω_c has impulse response

$$h_{\text{ideal}}[n] = \begin{cases} \frac{\omega_c}{\pi}, & n = 0, \\ \frac{\sin(\omega_c n)}{\pi n}, & n \neq 0. \end{cases}$$

It is two-sided and infinite. A length- $M + 1$ causal FIR shifts a symmetric section by $M/2$ and multiplies it by a window:

$$h[n] = h_{\text{ideal}}[n - M/2]w[n], \quad 0 \leq n \leq M.$$

Truncation causes Gibbs-like ripples. The window controls the compromise between transition width and sidelobe height. A Kaiser window is useful when you want a continuously adjustable parameter rather than a fixed named window. This method is transparent and often good enough; it does not produce the minimax-optimal filter for arbitrary weighted bands.

```
import numpy as np
from scipy import signal

fs = 48_000.0
numtaps = 161
cutoff = 8_000.0

h = signal.firwin(
    numtaps=numtaps,
    cutoff=cutoff,
    window="hann",
    fs=fs,
)
w, H = signal.freqz(h, worN=16_384, fs=fs)
passband = np.abs(H[w <= 7_000])
stopband = np.abs(H[w >= 10_000])
print("passband ripple (dB):", 20 * np.log10(passband.max() / passband.min()))
print("stopband attenuation (dB):", -20 * np.log10(stopband.max()))
```

The measured edges in this example are deliberately different from the design cutoff. A filter whose nominal cutoff is 8 kHz is not a brick wall at 8 kHz. Always measure the response against the actual passband and stopband limits.

7.5 Equiripple and Least-Squares FIR Design

The Parks–McClellan, or Remez, design treats the filter as a weighted approximation problem. In a typical low-pass design it minimises the maximum weighted error over passband and stopband. The resulting error alternates between positive and negative extrema of equal weighted magnitude. This equiripple property is a minimax certificate and usually gives a shorter filter than a simple windowed design for the same worst-case specification.

Least-squares design instead minimises an integrated squared error. It may give a smaller average error while allowing larger local peaks. The choice is a loss-function choice, not a matter of one algorithm being universally better. Weight bands according to the consequences of error. A stopband containing a dangerous interferer may need a much higher weight than a passband where a small ripple is harmless.

7.6 Phase, Group Delay, and Transients

For a frequency response $H(e^{j\omega}) = |H(e^{j\omega})|e^{j\phi(\omega)}$, group delay is

$$\tau_g(\omega) = -\frac{d\phi(\omega)}{d\omega}.$$

It is measured in samples when ω is radians per sample, and in seconds after division by f_s . Constant group delay preserves waveform alignment across frequencies. Rapidly varying delay can smear transients even when the magnitude response looks acceptable.

An FIR's step response reveals overshoot and settling behaviour. A filter with a narrow transition and high stopband attenuation often rings around sharp edges because a finite filter cannot be both sharply band-limited and compact in time. That ringing is not necessarily an implementation bug; it is the time-domain cost of the frequency-domain specification. The question is whether the ringing is acceptable for the signal and decision being made.

7.7 Analog Prototypes and IIR Designs

Classical IIR designs begin with an analogue low-pass prototype and map it to a digital filter. Butterworth filters have maximally flat magnitude in the passband. Chebyshev type I trades passband ripple for a sharper transition. Type II puts ripple in the stopband. Elliptic filters ripple in both and achieve the smallest order for given magnitude constraints, at the cost of more complicated phase and pole-zero geometry.

The bilinear transform is

$$s = \frac{2}{T} \frac{1 - z^{-1}}{1 + z^{-1}},$$

which maps the stable left half of the s -plane inside the unit circle. Its frequency mapping is nonlinear:

$$\Omega = \frac{2}{T} \tan\left(\frac{\omega}{2}\right).$$

To place a digital edge at ω_p , prewarp the analogue prototype edge to $\Omega_p = (2/T) \tan(\omega_p/2)$. If you skip prewarping at a high fraction of the sample rate, the realised cutoff can be materially displaced.

Impulse invariance maps analogue impulse-response samples into the digital filter. It preserves time-domain samples but replicates the analogue spectrum, so high-frequency analogue

content aliases. It is attractive for low-pass designs well below Nyquist and unsafe when the analogue prototype has meaningful energy near or above half the sampling rate.

7.8 Notches and Resonators Directly in the z -Plane

To make a notch at ω_0 , place zeros at $e^{\pm j\omega_0}$. To control the notch width, place poles at $re^{\pm j\omega_0}$ with $r < 1$:

$$H(z) = K \frac{1 - 2\cos(\omega_0)z^{-1} + z^{-2}}{1 - 2r\cos(\omega_0)z^{-1} + r^2z^{-2}}.$$

The zeros force exact cancellation at the target frequency in exact arithmetic. The nearby poles restore gain around the notch and determine selectivity. In finite precision, the zero angle and coefficient values are quantised, so the measured notch depth may be much shallower than the mathematical one. Normalize K at the frequency where unity gain matters; do not assume the unscaled numerator has the desired passband gain.

8 Implementing Filters: State, Precision, and Real-Time Constraints

A transfer function is not yet a production implementation. Realisations, state management, coefficient formats, and deadlines determine whether the mathematical filter survives contact with a stream and a processor.

8.1 Direct Forms and Biquad Cascades

A transfer function can be implemented in several algebraically equivalent ways. Direct Form I keeps separate input and output delay lines. Direct Form II combines them to reduce storage but may increase internal dynamic range. Transposed forms often have better numerical behaviour and map well to multiply-accumulate hardware.

For a high-order IIR, factor the denominator and numerator into first- and second-order sections:

$$H(z) = g \prod_{m=1}^K \frac{b_{0,m} + b_{1,m}z^{-1} + b_{2,m}z^{-2}}{1 + a_{1,m}z^{-1} + a_{2,m}z^{-2}}.$$

The biquad is the natural unit because complex poles and zeros occur in conjugate pairs for real filters. It limits the sensitivity of each section and makes section-by-section scaling

possible.

8.2 State Is Part of the Filter

For a streaming filter, the internal delay elements are state. Processing an array in one call and processing the same array in chunks should produce the same output if the state is carried between chunks. Resetting the state at every callback changes the system and often produces clicks, blocks of transient response, or incorrect low-frequency levels.

```
import numpy as np
from scipy import signal

fs = 48_000.0
sos = signal.butter(6, 2_000.0, btype="low", fs=fs, output="sos")
rng = np.random.default_rng(7)
x = rng.normal(size=100_000)

y_batch = signal.sosfilt(sos, x)
y_stream = np.empty_like(x)
state = signal.sosfilt_zi(sos) * x[0]

start = 0
block_sizes = (257, 1024, 89, 4096, 313)
block_index = 0
while start < len(x):
    block_size = block_sizes[block_index % len(block_sizes)]
    stop = min(start + block_size, len(x))
    y_stream[start:stop], state = signal.sosfilt(
        sos, x[start:stop], zi=state
    )
    start = stop
    block_index += 1

print("max batch/stream difference:", np.max(np.abs(y_batch - y_stream)))
```

The example uses a state initialisation that approximates steady state for a nonzero first sample. For a signal that is known to start at zero, zero state may be correct. There is no universally correct startup state; it encodes a model of what happened before the record began.

8.3 Finite Precision and Internal Dynamic Range

A filter can have a bounded external output and still overflow internally. Feedback sections may contain states much larger than the input or output, especially near resonance. Saturating arithmetic avoids wraparound's catastrophic sign changes but introduces nonlinear distortion. Scaling every section to keep its state inside range is safer than multiplying the entire cascade by one final gain.

Coefficient quantisation changes pole and zero locations. The sensitivity is high when poles are close together or close to the unit circle. Use a pole-zero calculation after quantisation,

excite the filter with an impulse and a full-scale signal, and measure the maximum internal state—not just the final output. On a fixed-point device, test all plausible input amplitudes and worst-case coefficient combinations.

A zero-input limit cycle is a persistent nonzero output caused by quantised feedback states. It is a direct demonstration that the finite-precision implementation is not the same linear system as the real-coefficient equation. Dither, dead-band logic, higher precision, or a different structure may be needed.

8.4 Offline Zero-Phase Filtering

Forward-backward filtering applies a causal filter to the record and then applies it backward. The phase contributions cancel in the interior, but the magnitude response is applied twice. If the one-pass response is $H(e^{j\omega})$, the idealised forward-backward magnitude is $|H(e^{j\omega})|^2$. The procedure is noncausal and edge handling determines how much of the record is contaminated by padding or startup assumptions.

Use it only when future samples are genuinely available and when the changed magnitude response is accounted for. It is not an acceptable substitute for specifying the latency of a real-time filter.

8.5 Real-Time Constraints Are Deadlines

A real-time DSP process does not merely need a low average runtime. It must finish each block before the next block’s deadline. If the audio callback receives 128 samples at 48 kHz, the nominal deadline is 2.667 ms, minus scheduling and I/O margin. One occasional 20 ms execution is a glitch even if the mean runtime is tiny.

Budget separately:

- **latency:** time from an input event to the corresponding output;
- **throughput:** sustained samples or frames per second;
- **memory:** coefficient, state, buffers, and temporary workspace;
- **jitter:** variation in execution or sampling time;
- **energy:** important for battery and thermally constrained devices;
- **precision:** enough bits for the required dynamic range and noise floor.

An FFT with good throughput can still violate latency if its block is too long. A sample-by-sample IIR can have tiny algorithmic latency but poor cache behaviour or an unstable fixed-point state. Profile the deployed structure and worst-case inputs.

8.6 Fixed-Point Representation

A signed fixed-point value with I integer bits and F fractional bits represents multiples of 2^{-F} over a finite range. Multiplying two such values produces a value with $2F$ fractional bits and a wider integer range; an implementation must choose where to round and how to rescale. The discarded low bits become quantisation noise, while discarded high bits become overflow or saturation.

Use guard bits in accumulators. A sum of K aligned values can require approximately $\lceil \log_2 K \rceil$ extra bits in the worst case. Random noise may not achieve the worst-case sum, but a production system should not rely on statistical cancellation when an adversarial or full-scale input is possible.

Saturation is usually safer than wraparound for signal amplitudes because it limits the error to a bounded excursion. It is nonlinear: once saturation occurs, superposition and spectral predictions no longer apply. Count and report saturation events rather than silently clipping.

8.7 Quantisation of Filter Coefficients

Suppose a second-order denominator is

$$1 + a_1 z^{-1} + a_2 z^{-2}.$$

Its pole locations are roots of $z^2 + a_1 z + a_2 = 0$. Quantising a_1 and a_2 perturbs those roots. For poles near the unit circle, a one-LSB coefficient change can move the resonance frequency or radius enough to alter bandwidth and decay time. Quantise first, then measure the actual response.

A coefficient set that looks well behaved in floating-point Python may be unusable on a 16-bit device. Export the quantised coefficients, run the same impulse and swept-sine tests through a bit-accurate model, and compare the pole-zero plot and internal-state ranges. The bit-accurate model is part of the design, not merely a test after deployment.

8.8 SIMD, Vectorisation, and Memory Layout

DSP is dominated by regular arithmetic and often benefits from single-instruction, multiple-data (SIMD) operations. The algorithm's mathematical complexity is only one part of performance. Contiguous memory, aligned loads, predictable branches, fused multiply-add, and avoiding unnecessary copies can matter more than a small algebraic simplification.

Array libraries vectorise across samples, channels, or filter sections differently. Check axis conventions before comparing performance. A vectorised operation over a two-dimensional array may allocate a temporary larger than the signal itself. On an embedded target, a hand-written circular-buffer kernel may be appropriate; on a workstation, a library implementation may use a more efficient blocked algorithm.

8.9 Block Processing and Partitioned Convolution

Long FIR filters are often implemented with overlap-save. Let the FFT size be N , the filter length be M , and the useful input block length be $L = N - M + 1$. Prepend the last $M - 1$ input samples to each block, transform, multiply by the filter transform, inverse-transform, and discard the first $M - 1$ output samples. The discarded samples are the circular-wrap portion; the remaining L samples agree with linear convolution.

Partitioned convolution splits a long impulse response into shorter partitions. Early partitions can use short FFTs to keep latency low, while later partitions use longer FFTs for efficiency. This is common for room impulse responses and long equalisation filters. The partition schedule is an engineering compromise among CPU load, memory, and latency.

8.10 Numerical Summation

An FIR output is a dot product. Adding many values with different magnitudes can lose small contributions in floating point. Pairwise summation or compensated summation improves accuracy when the dynamic range is large. In a typical audio filter, ordinary double precision is sufficient; in a long accumulation, high-order quadrature, or calibrated instrumentation path, the summation error may be comparable to the signal being measured.

Do not use a numerically elaborate summation method without measuring its cost and effect. Do use a known difficult test—alternating large and small terms, long white-noise records, and a reference computed at higher precision—when the error budget makes it relevant.

9 Sampling-Rate Conversion and Filter Banks

Changing the sample rate is itself a filtering problem. This chapter derives decimation, interpolation, rational conversion, polyphase efficiency, and the role of alias cancellation in filter banks.

9.1 Decimation and Interpolation

Decimation by M means low-pass filtering followed by retaining every M th sample:

$$y[n] = v[nM], \quad v = x * h.$$

The filter must suppress frequencies above the new Nyquist frequency $f_s/(2M)$ before samples are discarded. If it does not, the discarded high-frequency content aliases into the retained band.

Interpolation by L means inserting $L - 1$ zeros between samples, then low-pass filtering to remove the images created by the zero insertion. If $u[n]$ is the zero-stuffed sequence, the interpolation filter often has gain L in the passband so the original sample amplitudes are preserved:

$$y[n] = (h * u)[n], \quad H(e^{j0}) = L.$$

Whether a library applies this gain internally must be checked; a factor-of- L error can look like a harmless level change until it enters a calibrated or feedback system.

9.2 Rational Conversion

To convert by L/M , upsample by L , filter, and decimate by M . The intermediate rate is Lf_s , which can be much higher than either endpoint. Polyphase decomposition avoids calculating the zero-valued upsampled samples and avoids retaining samples that will be thrown away.

Split a prototype FIR into L phases:

$$h[r + Lp], \quad r = 0, \dots, L - 1.$$

Each output phase uses a different coefficient subsequence. The implementation computes only the phase needed for each output sample. This is not an approximation; it is a reordering of the same convolution sum.

For 48 kHz to 44.1 kHz, use $L/M = 147/160$. A direct implementation at the 7.056 MHz intermediate rate is wasteful; polyphase conversion performs roughly the necessary work at the input/output rates. In practice, a library such as `scipy.signal.resample_poly` designs or accepts an FIR window and applies the polyphase structure.

```
from scipy import signal
```

```
# 48 kHz -> 44.1 kHz. The output length is approximately len(x) * 147 / 160.
y = signal.resample_poly(x, up=147, down=160, window=("kaiser", 8.6))
```

The conversion filter’s passband and stopband must be specified relative to the lower of the two relevant Nyquist limits. Inspect the response and a swept-sine or multitone test; do not infer quality from the output length alone.

9.3 Multistage Conversion

Factor a large conversion into stages when intermediate filters become cheaper or easier to design. Half-band filters are useful for powers-of-two conversion because, aside from the centre coefficient, approximately half their coefficients can be zero. Cascaded integrator-comb (CIC) filters are multiplier-free and attractive in hardware, but their passband droop and compensation filter must be included in the design.

The right factorisation depends on the cost model: multiply cost, memory, clock domains, allowable latency, and whether the signal is real or complex. “One giant resampler” and “many small stages” are not universal answers.

9.4 Filter Banks

An analysis filter bank splits a signal into subbands; a synthesis bank recombines them. A critically sampled bank discards samples inside each subband, so alias cancellation and phase alignment must be designed into the analysis/synthesis pair. Perfect reconstruction means the cascade reproduces a delayed and scaled version of the input, not necessarily the input at zero delay.

This is the structural idea behind subband coding, wavelet filter banks, quadrature mirror filters, and many audio codecs. The deep lesson is that aliasing introduced inside a subband can cancel at the synthesis output if the bank’s algebra is designed for it; this does not make uncontrolled acquisition aliasing recoverable.

10 Random Signals and Spectral Measurement

For noisy or nonstationary data, a spectrum is an estimate, not a direct property read from an array. Start with the stochastic model, then define the PSD, window calibration, averaging, and uncertainty.

10.1 Processes and Realisations

A random process $X[n]$ is a collection of random variables indexed by time. One measured signal $x[n]$ is one realisation. The ensemble mean is $\mathbb{E}[X[n]]$; the autocorrelation is

$$R_{xx}[n_1, n_2] = \mathbb{E}[X[n_1]X^*[n_2]].$$

A process is wide-sense stationary (WSS) if its mean is constant and its autocorrelation depends only on lag:

$$R_{xx}[\ell] = \mathbb{E}[X[n]X^*[n - \ell]].$$

Stationarity is a model assumption, not a property granted by applying an FFT. A machine vibration signal during a speed ramp is not WSS over the whole record, even if a short segment is approximately stationary.

Whiteness means zero correlation at nonzero lag, or a flat power spectrum under the chosen model. Gaussianity means the finite-dimensional distributions are Gaussian. White noise need not be Gaussian; Gaussian noise need not be white. Uncorrelated random variables need not be independent unless additional distributional conditions apply.

10.2 Power Spectral Density

For a WSS process, the power spectral density (PSD) is the Fourier transform of the autocorrelation:

$$S_{xx}(e^{j\omega}) = \sum_{\ell=-\infty}^{\infty} R_{xx}[\ell]e^{-j\omega\ell}.$$

The inverse relation and the zero-lag identity give

$$R_{xx}[0] = \frac{1}{2\pi} \int_{-\pi}^{\pi} S_{xx}(e^{j\omega}) d\omega.$$

For an LTI filter, the output PSD is

$$S_{yy}(e^{j\omega}) = |H(e^{j\omega})|^2 S_{xx}(e^{j\omega}).$$

The square of the magnitude appears because power is a second-order quantity. If white noise has variance σ_x^2 , its discrete-time PSD is flat at σ_x^2 under the radians-per-sample convention. In hertz, the density depends on the sample-rate conversion and one-sided/two-sided convention.

10.3 Noise Bandwidth

A filter's equivalent noise bandwidth (ENBW) is the bandwidth of an ideal rectangular filter that would pass the same white-noise power for the same peak gain. For a discrete-time frequency response it is proportional to

$$\text{ENBW} = \frac{f_s \sum_n |h[n]|^2}{|\sum_n h[n]|^2}$$

for a finite impulse response with nonzero DC gain, using the usual hertz scaling. This explains why two filters with the same nominal cutoff can pass different amounts of white noise. A narrow-looking frequency response is not enough; integrate $|H|^2$.

10.4 What a Spectrum Estimate Means

An FFT of a finite record is a set of complex coefficients. A spectrum estimate is a statistical or deterministic interpretation of those coefficients with units and normalisation. Decide whether the result is intended to represent peak amplitude, RMS amplitude, power per bin, PSD in units squared per hertz, or amplitude spectral density in units per square-root hertz.

For a windowed record $x_w[n] = w[n]x[n]$, a common two-sided PSD periodogram in hertz is

$$\hat{S}_{xx}(f_k) = \frac{1}{f_s \sum_{n=0}^{N-1} w^2[n]} \left| \sum_{n=0}^{N-1} w[n]x[n]e^{-j2\pi kn/N} \right|^2.$$

The exact endpoint and one-sided factors depend on whether the input is real and whether the frequency grid includes Nyquist. The denominator is not optional: changing the window changes the noise power unless its mean-square energy is included.

10.5 Periodogram Bias and Variance

Windowing makes the periodogram's expected value a smoothed version of the true PSD: narrow features are broadened by the window's spectral energy. The ordinary periodogram remains noisy as record length increases; it is not a consistent pointwise estimator under common conditions because its variance does not vanish in the desired way.

The variance can be reduced by averaging estimates from shorter, often overlapping segments. The cost is frequency resolution and a loss of independent degrees of freedom when segments overlap. The correct segment length is tied to the stationarity interval and the feature width you need to resolve.

10.6 Welch's Method

Welch's method divides a record into windowed segments, computes a modified periodogram for each, and averages them. Averaging reduces the random fluctuations of the estimate and makes broad noise-floor comparisons much more stable. A 50% overlap is common for Hann windows because it recovers much of the data while maintaining useful averaging, but it is not a theorem that 50% is always optimal.

```
import numpy as np
from scipy import signal

fs = 10_000.0
rng = np.random.default_rng(42)
t = np.arange(0.0, 20.0, 1.0 / fs)
x = 0.4 * np.sin(2 * np.pi * 813.0 * t) + 0.1 * rng.normal(size=t.size)

f, psd = signal.welch(
    x,
    fs=fs,
    window="hann",
    nperseg=16_384,
    noverlap=8_192,
    detrend="constant",
    scaling="density",
)

tone_band = (f > 800) & (f < 830)
noise_band = (f > 2_000) & (f < 3_000)
print("peak tone frequency:", f[tone_band][np.argmax(psd[tone_band])])
print("noise power in band:", np.trapezoid(psd[noise_band], f[noise_band]))
```

The result has units of the squared input unit per hertz when `scaling="density"`. Integrating over a frequency band estimates mean-square power in that band, subject to leakage and estimator bias. Summing raw PSD bins without multiplying by bin width mixes power-per-hertz with power-per-bin.

10.7 Multitaper and Parametric Alternatives

Multitaper estimates use several orthogonal tapers, often discrete prolate spheroidal sequences, to average spectra while controlling leakage. They are useful when you need a better bias-variance trade-off than one window provides, especially for short records. The method introduces a time-bandwidth parameter and a choice of how to combine the tapered estimates.

Parametric autoregressive (AR) methods fit a model whose poles define a smooth spectrum. They can resolve close narrowband features with short records when the model is appropriate, but can invent peaks when the order or model family is wrong. A smooth-looking parametric spectrum is not automatically more truthful than a noisy nonparametric one.

10.8 Coherence and Cross-Spectra

The cross-power spectral density $S_{xy}(f)$ captures frequency-dependent correlation between two signals. Magnitude-squared coherence is

$$C_{xy}(f) = \frac{|S_{xy}(f)|^2}{S_{xx}(f)S_{yy}(f)}, \quad 0 \leq C_{xy}(f) \leq 1,$$

under valid PSD estimates. High coherence says that a linear relationship is stable across the averaged segments; it does not establish causation. Low coherence can mean nonlinear coupling, nonstationarity, independent noise, or inadequate signal-to-noise ratio.

11 Estimation, Detection, and Inverse Problems

Filtering can transform a signal, but it can also estimate an unknown quantity, detect a template, identify a system, or undo a measurement response. Those are objective- and noise-model-dependent problems.

11.1 Wiener Filtering

Suppose $d[n]$ is a desired signal and $x[n]$ is an observed input. A linear estimator

$$\hat{d}[n] = \sum_{k=0}^M w_k x[n-k]$$

can be chosen to minimise mean-square error $\mathbb{E}[d[n] - \hat{d}[n]]^2$. Differentiating with respect to the coefficient vector gives the Wiener–Hopf equations

$$R_{xx}w = p_{xd},$$

where $R_{xx} = \mathbb{E}[xx^H]$ and $p_{xd} = \mathbb{E}[xd^*]$. The solution is $w = R_{xx}^{-1}p_{xd}$ when the correlation matrix is nonsingular. In practice, estimate these quantities from finite data and solve the system without explicitly forming an inverse. Regularisation may be required if R_{xx} is ill-conditioned.

The Wiener filter is optimal only for the stated linear, mean-square objective and the assumed joint statistics. It is not universally optimal for perceptual quality, outlier robustness, or nonstationary signals.

11.2 Matched Filtering

For a known deterministic pulse $s[n]$ in additive white Gaussian noise, the matched filter has impulse response proportional to $s^*[-n]$. Its output samples the correlation of the data with the expected pulse. The peak gives the candidate delay, and the peak-to-noise distribution determines a detection threshold.

In coloured noise, a plain correlation is not generally optimal. Whiten the noise first or use a filter whose frequency response weights the template by the inverse noise PSD. A strong periodic interferer can create multiple correlation peaks; a high peak is evidence only relative to the noise and template model.

11.3 Deconvolution

If an observation is $y = h * x + v$, where v is noise, the frequency-domain inverse estimate

$$\hat{X}(e^{j\omega}) = \frac{Y(e^{j\omega})}{H(e^{j\omega})}$$

is unstable wherever $|H|$ is small. A Wiener-style regularised inverse replaces the exact division with a denominator containing an estimate of the signal and noise spectra, for example

$$\hat{X}(e^{j\omega}) = \frac{H^*(e^{j\omega})}{|H(e^{j\omega})|^2 + \lambda(\omega)} Y(e^{j\omega}).$$

The regulariser accepts bias to control variance. A sharper deblurred image or waveform is not automatically more accurate; inspect residuals and uncertainty.

11.4 Least-Mean-Squares Adaptation

An adaptive FIR filter updates its weights as data arrive. With input vector $u[n]$, desired response $d[n]$, estimate $y[n] = w^H[n]u[n]$, and error $e[n] = d[n] - y[n]$, the least-mean-squares (LMS) update is

$$w[n+1] = w[n] + \mu e^*[n]u[n].$$

The update is a stochastic approximation to the negative gradient of mean-square error. The step size μ controls convergence speed and misadjustment. A rough stability condition depends on the largest eigenvalue of the input correlation matrix; highly correlated inputs require a smaller step size than white inputs with the same variance.

Normalised LMS divides the update by $\epsilon + \|u[n]\|^2$, making it less sensitive to input level. Recursive least squares converges faster for some problems but costs more and is more sensitive to numerical and forgetting-factor choices.

11.5 System Identification

To identify an unknown LTI system, excite it with a signal whose spectrum covers the frequencies of interest, record input and output synchronously, and estimate a model. A single sine identifies one frequency. An impulse is conceptually broad but can have poor signal-to-noise ratio. Pseudorandom sequences and swept sines provide controlled excitation.

The experiment must separate excitation from noise and account for delay, sensor dynamics, clipping, and nonstationarity. Fitting a high-order model to a quiet, narrowband record can produce a model that predicts the record but says little about the system outside it.

11.6 Stable Inversion as Regularised Projection

If a filter strongly attenuates a subspace of signals, inversion cannot recover that subspace robustly. In a least-squares formulation,

$$\hat{x} = \arg \min_x \|Hx - y\|_2^2 + \lambda \|Lx\|_2^2,$$

where L encodes a smoothness or prior penalty. The normal equations are

$$(H^H H + \lambda L^H L) \hat{x} = H^H y.$$

This is the DSP form of regularisation encountered in numerical analysis. The parameter λ controls bias-variance trade-off; choose it from a noise model, validation experiment, discrepancy principle, or an explicitly stated engineering criterion.

12 Applications: Time-Varying, Complex, and Spatial Signals

The core machinery transfers to signals that are local in time, naturally complex, or indexed over space. These are applications of the same models, not separate kinds of mathematics.

12.1 The Short-Time Fourier Transform

The short-time Fourier transform (STFT) applies a window around each time position:

$$X(m, \omega) = \sum_n x[n]w[n - mR]e^{-j\omega n},$$

where R is the hop size. The spectrogram is usually $|X(m, \omega)|^2$. A long window gives fine frequency discrimination and poor timing; a short window gives better transient timing and broader frequency lobes. This is a consequence of the time-frequency uncertainty trade-off, not a software limitation.

The window, hop, FFT length, centering convention, and boundary padding all affect the picture. FFT zero-padding can make each frame's displayed frequency curve smoother without improving the window's ability to separate nearby components.

For reconstruction, analysis and synthesis windows must satisfy an overlap-add condition. A spectrogram is not automatically invertible just because it looks plausible. Libraries may normalise or pad frames differently; test reconstruction with an impulse, a constant signal, and a chirp.

12.2 Analytic Signals and the Hilbert Transform

The analytic signal associated with a real signal $x[n]$ is

$$z[n] = x[n] + j\mathcal{H}\{x[n]\},$$

where the Hilbert transform shifts positive and negative frequency components by opposite phases and, in the ideal construction, removes negative frequencies from the complex result. The envelope is $|z[n]|$ and a wrapped instantaneous phase is $\arg z[n]$.

Instantaneous frequency from an unwrapped phase derivative is meaningful for a narrowband or well-separated-component signal. For a sum of unrelated tones, the envelope can approach zero and the phase can jump; the derivative then has no simple physical interpretation. The analytic signal is a tool with conditions, not a universal way to label every oscillation.

12.3 Example: Chirps and Transients

A chirp whose frequency changes over time spreads along a curve in a spectrogram. A transient contains broad frequency content for a short time, so a narrowband long-window spectrogram smears it. Use a collection of window lengths or a transform designed for the signal's structure, and state whether the output is qualitative or used for measurement.

For a detection problem, a matched filter or time-domain template may be more interpretable than a spectrogram. For a slowly varying harmonic process, an STFT or sinusoidal tracker may be appropriate. Method selection follows the signal model.

12.4 Why Complex Signals Appear

Complex-valued sequences are not a claim that a sensor measures imaginary voltage. They are a compact representation of two real channels, or of amplitude and phase relative to a reference oscillator. In a quadrature receiver, the in-phase (I) and quadrature (Q) components are mixed down with cosine and sine local oscillators and then low-pass filtered. The complex baseband sequence

$$z[n] = I[n] + jQ[n]$$

retains signed frequency information: a positive-frequency component and a negative-frequency component rotate in opposite directions in the complex plane. A real low-pass signal cannot distinguish those directions because its spectrum is conjugate symmetric; I/Q sampling can.

The representation only works if the I and Q paths are calibrated. Gain mismatch, phase error, local-oscillator leakage, and timing skew create an unwanted image on the opposite side of baseband. A complex signal can therefore make a receiver's failure modes more visible, not make them disappear.

12.5 Modulation as Frequency Translation

Multiplying by a complex exponential shifts a spectrum:

$$x[n]e^{j\omega_0 n} \iff X(e^{j(\omega-\omega_0)}).$$

Multiplying a real signal by a cosine creates two shifted copies, one at $+\omega_0$ and one at $-\omega_0$. Multiplying by a complex exponential creates one directional shift. A receiver mixes a bandpass signal down to a lower intermediate frequency or to baseband, filters the desired band, and may decimate after the band has been isolated.

The order matters. If the input is sampled at 100 MHz and contains a wanted 10 MHz-wide band around 20 MHz, mixing it to baseband is not enough: the low-pass filter still has to reject neighbouring channels before decimation. Decimating first may alias the neighbouring

channels into the wanted band.

12.6 Amplitude, Phase, and Frequency Modulation

In amplitude modulation, information changes the envelope of a carrier. In phase modulation, it changes the carrier's phase; in frequency modulation, the instantaneous frequency changes. These descriptions are related through differentiation and integration of phase, but the demodulator's noise and filter requirements differ.

For a complex envelope $a[n]e^{j\phi[n]}$, amplitude is $a[n]$ and phase is $\phi[n]$. An envelope detector based on $|z[n]|$ fails when the signal has multiple components or when the analytic-signal assumption does not hold. Phase demodulation requires unwrapping or a differential phase operation, and large phase jumps can be indistinguishable from rapid frequency changes without a bandwidth constraint.

12.7 Matched Filtering and Detection Thresholds

Let the observation be

$$y[n] = \alpha s[n - n_0] + v[n],$$

where s is a known template, α is an unknown amplitude, and v is noise. The matched-filter output at lag ℓ is proportional to

$$r_{sy}[\ell] = \sum_n s^*[n]y[n + \ell].$$

Under white Gaussian noise, the output signal-to-noise ratio is maximised when the filter is matched to the template. If the noise covariance is not proportional to the identity, the optimum metric weights the observation by the inverse covariance, which is equivalent to whitening before correlation.

Detection requires a threshold and a decision policy. A threshold chosen by looking at the same data being tested produces optimistic false-alarm estimates. Estimate the noise distribution from a clean reference or use a held-out time interval. For a search over many lags, the maximum of many noise-only correlation values has a larger false-alarm probability than one individual value; the multiple-comparisons effect must be included.

12.8 IQ Imbalance and Image Rejection

An ideal complex mixer produces I and Q with exactly 90 degrees of phase difference and equal gain. Let the gain and phase errors be small. The resulting complex signal can be approximated as

$$z_{\text{meas}}[n] \approx \alpha z[n] + \beta z^*[n].$$

The conjugated term is the image. Calibration can estimate α and β using a known complex tone and compensate them, but the compensation itself must be stable across frequency and temperature. Inspect image rejection with a tone whose positive- and negative-frequency locations are unambiguous.

12.9 Two-Dimensional Convolution

For an image $x[m, n]$ and kernel $h[i, j]$, two-dimensional convolution is

$$y[m, n] = \sum_i \sum_j h[i, j] x[m - i, n - j].$$

Separable kernels factor as $h[i, j] = a[i]b[j]$, allowing one horizontal and one vertical pass rather than a full two-dimensional operation. Gaussian smoothing is approximately separable; many edge operators are designed as separable or near-separable kernels.

Boundary policy is part of the filter: zero padding creates dark or low-valued borders, reflection preserves local continuity better for some images, and circular boundaries are appropriate only when periodicity is intended. FFT image filtering implicitly uses circular boundaries unless padding is handled.

12.10 The 2-D Fourier Transform

The two-dimensional DFT decomposes an image into spatial frequencies. Low spatial frequencies describe broad illumination and smooth structure; high frequencies describe edges and fine texture. A circularly symmetric low-pass mask smooths but can ring near sharp edges because of the same finite-support trade-off as one-dimensional filters.

The separability and convolution theorem extend to dimensions, but so do the traps: the transform origin, `fftshift`, normalization, frequency units per pixel, and boundary assumptions must be explicit. A visually pleasing filtered image is not evidence that the operation preserved photometric quantities.

12.11 Sampling in Space

Image aliasing appears as moiré, jagged edges, and false textures when spatial frequencies exceed the pixel grid's representable band. Anti-aliasing during resizing requires low-pass filtering before decimation in each dimension. Sharpening before downsampling can make aliasing worse; filtering after downsampling cannot generally recover the lost distinction.

13 Building and Verifying a DSP Pipeline

Close with the engineering loop: define the question, choose a method, quantify errors, test invariants, and compare a controlled pipeline with the deployed one.

13.1 First Questions

Before choosing an FFT, filter, or resampler, answer:

- What is measured, and in what physical units?
- Which frequencies or time events carry the information?
- Is the signal stationary over the record or only locally?
- Is future data available?
- What error, latency, throughput, and memory limits apply?
- Is the output for display, estimation, control, detection, storage, or reconstruction?
- Which assumptions are testable from the data and metadata?

If the answer is “I need to see what is there,” start with a calibrated, windowed exploratory spectrum and a time-domain plot, but label it exploratory. If the answer is “estimate power,” use a PSD method and integrate it. If the answer is “remove a known band,” design a filter against explicit edges and attenuation. If the answer is “detect a known waveform,” begin with correlation or matched filtering and model the noise.

13.2 Method Selection Heuristics

Need	First method to evaluate	Main risk
Smooth sensor display	low-order FIR or IIR low-pass	hidden latency and edge/startup transients
Calibrated phase/amplitude	linear-phase FIR or characterised IIR	delay, order, and passband calibration
Narrow known interference	notch or band-stop	ringing, pole sensitivity, drifting interference

Need	First method to evaluate	Main risk
Large convolution	overlap-save/overlap-add FFT	circular-boundary errors and block latency
Lower sample rate	anti-alias FIR plus polyphase conversion	insufficient stopband rejection
Broadband noise level	Welch or multitaper PSD	incorrect units and estimator uncertainty
Known pulse in noise	matched filter/correlation	coloured noise and ambiguous peaks
Time-varying frequency	STFT or model-based tracker	window-dependent resolution
Unknown system response	designed excitation plus system identification	poor excitation coverage

These are starting points. Measure against the real acceptance criterion and keep the simplest method that meets it.

13.3 A DSP Error Budget

A reliable pipeline separates at least these error sources:

1. sensor and analogue noise;
2. analogue distortion and anti-alias-filter leakage;
3. timing error and sample-rate drift;
4. ADC/DAC quantisation and converter nonlinearity;
5. model approximation, such as finite filter order;
6. finite-record and estimator uncertainty;
7. floating-point or fixed-point arithmetic;
8. boundary, state, and calibration errors.

If a measured effect is smaller than the combined uncertainty, reporting many decimal places does not make it real. If two implementations disagree, first compare units, sample rate, window, delay, initial state, and normalization before comparing algorithms.

13.4 Properties Worth Testing

For a custom implementation, test invariants rather than only a few expected arrays:

- impulse response against the coefficients;

- constant input against DC gain;
- complex sinusoid against the predicted $H(e^{j\omega})$;
- linearity with two inputs and arbitrary scalar coefficients;
- time invariance away from explicitly tested boundaries;
- batch/streaming equivalence with irregular block lengths;
- Parseval energy agreement under the documented FFT normalization;
- reconstruction after upsample/filter/downsample or STFT analysis/synthesis;
- PSD integration against time-domain variance;
- alias rejection using tones just inside and outside the intended band;
- long-run state behaviour with zero input and full-scale input.

Property tests catch sign conventions and boundary mistakes that a single plot can hide.

13.5 Reproducibility

Record the sample rate, units, channel order, time origin, filter coefficients, design method, window, overlap, FFT length, scaling convention, library versions, and random seed. Store calibration constants and whether a plot is one-sided or two-sided. If a resampler is used, record the exact rational ratio and prototype filter.

An output named `spectrum.png` is not a reproducible result. A reproducible result includes the input, code, parameters, and interpretation needed to regenerate its axes and units.

13.6 Common Failure Patterns

Filtering after aliasing. The digital filter is applied after an ADC has already folded out-of-band energy into the signal band. Fix the acquisition chain or sample faster with appropriate analogue filtering.

Calling bin spacing resolution. A zero-padded FFT has more plotted bins but not a longer observation. Use a longer record, a suitable estimator, or a model-based frequency estimator if the application needs better discrimination.

Using raw FFT magnitude as a PSD. The result lacks window, sample-rate, sidedness, and amplitude calibration. State the desired units and use a tested scaling formula or library method.

Resetting IIR state at block boundaries. The output contains repeated startup transients. Carry state, or explicitly discard and overlap enough samples to make the boundary policy part of the design.

Implementing a high-order IIR with one polynomial. Coefficient rounding and cancellation move poles and amplify roundoff. Use second-order sections and inspect the quantised structure.

Assuming filtfilt is a real-time filter. It uses future samples, doubles the magnitude response, and depends on edge padding. Use a causal design when the output must be available online.

Treating a smooth spectrum as proof. Smoothing can suppress variance and hide narrow features. Report segment length, window, overlap, resolution, and uncertainty.

Using one window for every objective. Windows optimise different leakage, resolution, amplitude, and noise-bandwidth compromises. Choose and calibrate one for the measurement.

Ignoring units. Radians per sample, hertz, samples, seconds, amplitude, power, and density are not interchangeable. Put units on axes and in variable names.

The following example estimates a vibration spectrum, removes a known electrical interference, and checks its output in both time and frequency. The numerical values are illustrative; an engineering analysis must substitute the sensor calibration and acceptance limits.

13.7 Generate a Controlled Test Signal

```
import numpy as np
from scipy import signal

rng = np.random.default_rng(2026)
fs = 12_000.0
duration = 8.0
t = np.arange(0.0, duration, 1.0 / fs)

# A wanted vibration tone, an interference tone, and broadband noise.
x = (
    0.8 * np.sin(2 * np.pi * 730.0 * t)
    + 0.25 * np.sin(2 * np.pi * 2_400.0 * t + 0.4)
    + 0.08 * rng.normal(size=t.size)
)
```

The test has known truth: 730 Hz and 2.4 kHz tones, plus noise with known variance. It is deliberately not an integer number of cycles in every likely segment, so the spectral estimator has to handle leakage.

13.8 Design and Apply a Notch

```
f0 = 2_400.0
quality = 35.0
b, a = signal.iirnotch(w0=f0, Q=quality, fs=fs)
sos = signal.tf2sos(b, a)
```

```
# Offline result; a live system would use sosfilt and accept phase/latency.
y = signal.sosfiltfilt(sos, x)
```

The notch is narrow, so it will not remove broadband noise around the interference. `sosfiltfilt` removes interior phase distortion but has edge behaviour and a squared magnitude response. For a causal monitoring path, use `sosfilt`, carry its state across blocks, and document the group delay.

13.9 Estimate and Integrate the PSD

```
f, pxx = signal.welch(
    y,
    fs=fs,
    window="hann",
    nperseg=16_384,
    noverlap=8_192,
    detrend="constant",
    scaling="density",
)

def band_power(frequency, density, lo, hi):
    mask = (frequency >= lo) & (frequency <= hi)
    return np.trapezoid(density[mask], frequency[mask])

print("730-Hz band power:", band_power(f, pxx, 700, 760))
print("2.4-kHz band power:", band_power(f, pxx, 2_350, 2_450))
print("total estimated variance:", np.trapezoid(pxx, f))
print("time-domain variance:", np.var(y))
```

The integrated PSD and time-domain variance should be close, subject to the detrending, finite-band integration, window bias, and the fact that `sosfiltfilt` changes the record at its boundaries. The 2.4 kHz band should be substantially reduced, but “zero” is not expected: the notch has finite numerical depth, the tone is not necessarily exactly represented by the estimator’s bins, and surrounding noise remains.

13.10 Validate Against a Causal Block Path

```
state = signal.sosfilt_zi(sos) * x[0]
chunks = []
start = 0
block_sizes = (511, 2048, 73, 997)
block_index = 0
while start < len(x):
    block_size = block_sizes[block_index % len(block_sizes)]
    stop = min(start + block_size, len(x))
    block, state = signal.sosfilt(sos, x[start:stop], zi=state)
    chunks.append(block)
    start = stop
    block_index += 1

y_causal = np.concatenate(chunks)
print("causal output length:", len(y_causal))
```

Do not compare `y_causal` sample-for-sample with `y`, because one is causal and phase-distorting while the other is noncausal and zero-phase. Compare the metric each path is

intended to preserve: notch attenuation, tone amplitude, latency, edge behaviour, and accepted phase response.

14 Reference Appendix

Use this section when implementing or reviewing a pipeline. The main chapters explain the ideas; these tables and checklists collect the details most often omitted in practice.

14.1 Complex Arithmetic Without Mysticism

Write a complex number as $z = a + jb$, where $j^2 = -1$. Its conjugate is $z^* = a - jb$, magnitude is $|z| = \sqrt{a^2 + b^2}$, and phase is $\arg z$. The product of two complex numbers multiplies magnitudes and adds phases. Euler’s identity,

$$e^{j\theta} = \cos \theta + j \sin \theta,$$

turns sinusoidal algebra into multiplication. The real part of $Ae^{j\theta}$ is a cosine with amplitude $|A|$ and phase $\arg A + \theta$.

Complex conjugation in a transform is not optional notation. For complex data, the energy is $|x[n]|^2 = x[n]x^*[n]$, not $x[n]^2$. Inner products conjugate one argument, and correlation conjugates the template. Accidentally using a transpose instead of a conjugate transpose produces wrong power and often wrong negative-frequency behaviour.

14.2 Decibels and Reference Quantities

For a power or power-like ratio,

$$L_{\text{dB}} = 10 \log_{10} \frac{P}{P_0}.$$

For an amplitude ratio when power is proportional to amplitude squared,

$$L_{\text{dB}} = 20 \log_{10} \frac{A}{A_0}.$$

The denominator is part of the measurement. “−60 dB” is meaningless without saying relative to what: full scale, a carrier, 1 volt RMS, 1 g RMS, or the noise floor. A filter magnitude of 0.001 is −60 dB in amplitude and −120 dB in power.

RMS amplitude of a zero-mean sinusoid of peak amplitude A is $A/\sqrt{2}$. A one-sided amplitude spectrum doubles positive-frequency interior components, while a one-sided power spectrum doubles power. Applying both a power and an amplitude doubling is a common 3 dB error.

14.3 Frequency Units

The same frequency can be expressed as:

$$f \text{ Hz}, \quad \Omega = 2\pi f \text{ rad/s}, \quad \omega = 2\pi f/f_s \text{ rad/sample}, \quad \nu = f/f_s \text{ cycles/sample}.$$

The digital frequency response is periodic in ω with period 2π and in ν with period 1. A filter specified at 2 kHz has no meaning until the sample rate is known. A normalised cutoff of 0.2 in a library may mean 0.2 times the Nyquist frequency, 0.2 cycles/sample, or radians/sample depending on the API. Prefer passing `fs=` where supported and label all plots.

14.4 Converting Ripple and Attenuation to Linear Values

If the passband magnitude must lie within R_p dB of unity, the lower linear bound is

$$\delta_p = \frac{1 - 10^{-R_p/20}}{1 + 10^{-R_p/20}}$$

under one common symmetric-ripple parameterisation. If the stopband attenuation must be at least A_s dB, the maximum linear stopband magnitude is

$$\delta_s = 10^{-A_s/20}.$$

The exact order formula depends on the filter family and frequency transformation, but the design process always compares a desired response to an achieved response in a specified norm. Convert dB constraints carefully; a 6 dB amplitude error is roughly a factor of two, while a 6 dB power error is also roughly a factor of four in power.

14.5 FIR Order Intuition

For a windowed-sinc FIR, order grows approximately inversely with normalized transition width. If the sample rate doubles while the physical transition width stays fixed, the normalized transition narrows and the required number of taps rises. This is why multirate systems

often filter after an early rate reduction when the signal band permits it.

Kaiser-window estimates commonly take the form

$$N \approx \frac{A - 8}{2.285 \Delta\omega},$$

where A is desired attenuation in dB and $\Delta\omega$ is transition width in radians/sample. The estimate is a starting point, not the achieved specification. Evaluate `freqz` on a sufficiently dense grid and measure the actual extrema. A coarse grid can miss a narrow ripple peak and falsely certify a filter.

14.6 IIR Order and Warp Checks

An analogue prototype order function usually works with prewarped passband and stopband frequencies. The resulting digital response must still be measured after the bilinear transform. At low normalized frequencies the tangent mapping is nearly linear; near Nyquist it is strongly warped. A design that looks correct in an analogue frequency table can miss its digital edge if the conversion is applied incorrectly.

Always check:

1. the prototype's frequency units;
2. whether the API expects a half-cycle-normalized cutoff or hertz;
3. whether prewarping is internal or must be supplied;
4. coefficient ordering and feedback sign convention;
5. pole radii after quantisation;
6. response, delay, and impulse/step behaviour at the deployed sample rate.

14.7 Filter Diagnostics

Use several diagnostics because no one view catches every problem:

- `freqz` or an equivalent dense response for magnitude and phase;
- unwrapped phase and group delay for timing distortion;
- an impulse response for coefficient order and stability;
- a step response for overshoot and settling;
- a logarithmic swept sine for audible or measurable passband/stopband defects;
- white-noise PSD before and after filtering for the $|H|^2$ relationship;
- pole-zero geometry for resonances and minimum-phase claims;
- max internal state under full-scale and crest-factor-heavy signals.

The diagnostics should be run on the exact coefficient format and exact structure used in production. A floating-point transfer function is not a substitute for a fixed-point cascade test.

Before publishing a spectrum, record:

1. sample rate and physical input units;
2. record duration and number of samples;
3. whether the mean or trend was removed;
4. window type and coherent-gain or power correction;
5. FFT length and whether zero-padding was used;
6. one-sided/two-sided convention;
7. amplitude, RMS, power, PSD, or amplitude-density units;
8. segment length, overlap, and averaging method;
9. treatment of DC and Nyquist bins;
10. uncertainty or an explanation of estimator variability.

Before changing the sample rate, record:

1. the original and target rates;
2. the desired retained band;
3. the required alias/image attenuation;
4. transition bands at the intermediate rate;
5. passband gain and allowable ripple;
6. group delay and boundary policy;
7. whether the conversion must be causal and streaming;
8. the rational ratio or clock-tracking strategy;
9. output-length and timestamp convention;
10. a swept-tone test near every relevant edge.

The checklists are intentionally mundane. Most production DSP failures are not failures to derive a Fourier transform; they are undocumented assumptions about units, boundaries, state, or calibration.

15 Further Reading and Resources

The following references cover the subject more fully and provide the derivations behind the shortcuts used here.

- [MIT OpenCourseWare: Digital Signal Processing, RES.6-008](#) — a compact graduate-level sequence covering discrete-time systems, convolution, the z -transform, the DFT, filter design, and the FFT.
- [MIT OpenCourseWare: Signals and Systems, 6.003](#) — useful foundations for continuous-time Fourier analysis, sampling, modulation, and the relationship between continuous and discrete representations.
- Oppenheim, Schafer, and Buck, *Discrete-Time Signal Processing* — the main reference for classical discrete-time DSP.
- Smith, **The Scientist and Engineer's Guide to Digital Signal Processing* — an accessible companion for intuition and practical examples; use a formal text for proofs and exact specifications.
- Smith, *Spectral Audio Signal Processing* — especially useful for windows, spectral measurement, and STFT analysis/synthesis.
- Cooley and Tukey, “An algorithm for the machine calculation of complex Fourier series” — the classic FFT paper. The FFT computes the DFT; it does not define a new transform.
- Parks and McClellan, “Chebyshev approximation for nonrecursive digital filters with linear phase” — original reference for equiripple linear-phase FIR design.
- Vaidyanathan, “Multirate digital filters, filter banks, polyphase networks, and applications” — a substantial tutorial on multirate structures and filter banks.
- [NumPy FFT reference](#) — array conventions, real transforms, frequency helpers, and normalization.
- [SciPy signal reference](#) — filter design, filtering, resampling, spectral analysis, and analytic-signal routines.
- Welch, “The use of fast Fourier transform for the estimation of power spectra” — original reference for the averaged modified-periodogram method.

The durable habit to take from DSP is simple: model the signal chain, write down the transform convention, and verify the result against an identity or a controlled experiment. The mathematics gives you the relationships; the engineering work is making sure the conditions for those relationships are actually true.